

1. Programació amb el DOM (I)

En aquesta primera lliçó descobrim què és exactament el DOM, com s'organitza en forma d'arbre i com podem localitzar-hi elements amb les primeres eines de JavaScript que farem servir (com `getElementById` i `querySelector`). Aprenem a llegir i canviar el text i alguns atributs bàsics dels nodes, a diferenciar entre propietats com `textContent`, `innerText` i `innerHTML`, i ho posem en pràctica amb exemples senzills perquè veiem com, pas a pas, el nostre codi pot començar a transformar el contingut d'una pàgina web.

1.1 Estructura del DOM

El DOM (*Document Object Model*) és una representació en forma d'**arbre jeràrquic** de tots els elements que componen una pàgina HTML. Cada etiqueta, text o atribut es converteix en un **node** dins d'aquesta estructura, i tots aquests nodes s'organitzen en relacions de **pare**, **fills** i **germans**. Aquesta representació no és el codi HTML en si mateix, sinó un **model intern que manté el navegador a memòria**, sobre el qual podem treballar mitjançant JavaScript. Gràcies al DOM podem **accedir, modificar, afegir o eliminar elements i contingut de la pàgina en temps real**, fet que permet crear interfícies web dinàmiques i interactives.

1.1.1 Què és el DOM?

El **DOM** (*Document Object Model*) és la representació en forma d'objectes del document HTML (o XML) que carrega el navegador:

- El navegador llegeix el fitxer HTML.
- El transforma en un **arbre de nodes**.
- Cada etiqueta, text, comentari... es converteix en un **node**.
- JavaScript pot accedir i modificar aquests nodes durant l'execució.

Quan parlem de “treballar amb el DOM” estem parlant, en realitat, de treballar amb **objectes** i les seves **propietats/mètodes**:

```
1 <!DOCTYPE html>
2 <html lang="ca">
3   <head>
4     <meta charset="utf-8">
```

```
5    <title>Exemple DOM</title>
6  </head>
7  <body>    <h1 id="titol">Hola DOM</h1>
8            <p class="resum">Un petit exemple</p>
9  </body>
10 </html>
```

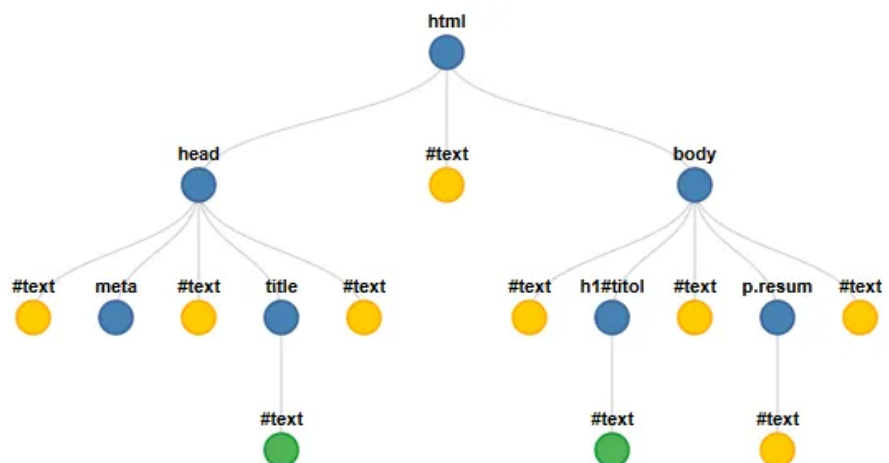
Aquest codi es representa internament com un **arbre de nodes**:

- document (arrel)
 - html
 - * head
 - meta
 - title → node de text “Exemple DOM”
 - * body
 - h1 (id=“titol”) → node de text “Hola DOM”
 - p (class=“resum”) → node de text “Un petit exemple”

Per tant, podem dir que el DOM no és el codi HTML, sinó l'**estructura d'objectes** que el navegador construeix a partir d'aquest codi i la relació que tenen aquests objectes.

Si dibuixem tots els nodes de l'exemple anterior obtenim (figura 3.1):

FIGURA 1.1. Estructura de l'arbre del DOM



En aquesta imatge (que és un llistat de tots els nodes de l'exemple) veiem nodes text que a primera vista sembla que no existeixin en el nostre DOM. Això és perquè els salts de línia, espais en blanc..., són comptats com a nodes text, encara que visualment no hi hagi res.

1.1.2 Tipus de models DOM

Històricament, s'han definit especificacions del DOM (DOM Level 1, 2, 3 i l'actual DOM Living Standard / DOM4). En la pràctica actual:

- Treballarem amb **HTML DOM**, que és el model per a documents HTML.
- El mateix concepte s'aplica a altres formats (XML DOM, SVG DOM...), però en aquest mòdul ens centrarem en HTML.

Els navegadors moderns implementen de manera força homogènia les parts del DOM que utilitzarem al curs.

1.1.3 Interfícies principals del DOM

El DOM està definit com un conjunt d'**interfícies** (tipus d'objectes JavaScript) que descriuen què pot fer cada mena de node dins de l'arbre. Cada interfície agrupa **propietats i mètodes** comuns (per exemple, com accedir als fills, llegir o modificar el text, gestionar atributs...) i moltes d'elles **hereten** d'una interfície base anomenada Node. D'aquesta manera, elements, textos, comentaris i el document en si comparteixen un comportament comú, però al mateix temps poden afegir funcionalitats específiques segons el seu tipus:

- **Node:** Interfície base de la qual hereten molts altres tipus (Document, Element, Text...). Proporciona propietats comunes (com parentNode, childNodes...).
- **Document:** Representa tot el document. És el punt d'entrada principal al DOM (document). Permet **cercar** i **crear** nodes nous.
- **Element:** Representa una etiqueta HTML (<p>, <div>, ...). Afegeix propietats i mètodes específics dels elements (atributs, classes, estils...).
- **Text:** Representa el contingut textual dintre d'un element.
- **Altres interfícies** (que veurem més endavant o de manera puntual): Attr, Comment, DocumentFragment, HTMLDocument...

1.1.4 L'arbre de nodes: pares, fills i germans

L'estructura del DOM es pot imaginar com un **arbre jeràrquic** on tots els elements HTML queden connectats entre si. A la part superior trobem un **node arrel** (document) del qual pengen la resta de nodes: l'element <html>, que al seu

torn conté `<head>` i `<body>`, i així successivament fins a arribar als nodes més petits, com ara paràgrafs, enllaços o fragments de text (les **fulles** de l'arbre).

Per poder treballar amb aquesta estructura és important conèixer alguns **conceptes bàsics** com ara *pare* (el node que conté un altre node), *fill* (el node contingut), *germans* (nodes que comparteixen el mateix pare) o *subarbre* (una part de l'arbre completa dins d'un altre node).

Aquests conceptes ens ajudaran a entendre com es poden **recórrer i modificar** les diferents parts del DOM amb JavaScript:

- **Node pare:** el node que conté un altre node.
- **Node fill:** un node contingut directament dins d'un altre.
- **Nodes germans:** nodes que comparteixen el mateix pare.
- **Descendent:** qualsevol node que estigui “per sota” (no cal que sigui fill directe).

Per exemple:

```
1 <ul>
2   <li>Primer</li>
3   <li id="central">Segon</li>
4   <li>Tercer</li>
5 </ul>
```

On les relacions són:

- El `<ul>` és el **pare** de tots els `<li>`.
- Cada `<li>` és **fill** del `<ul>`.
- Els tres `<li>` són **germans** entre si.

1.1.5 Informació bàsica dels nodes: `nodeType` i `nodeName`

La interfície `Node` proporciona propietats bàsiques per **identificar i descriure** cada node de l'arbre del DOM. Entre aquestes propietats n'hi ha dues d'especialment importants: `nodeType` i `nodeName`. Amb `nodeType` el navegador indica, mitjançant un **codi numèric**, de quin tipus de node es tracta (element, text, comentari, document...), mentre que `nodeName` ens dona el **nom del node** en forma de cadena (per exemple “P” per a un `<p>` o “#text” per a un node de text). Conèixer aquests valors ens permet **distingir clarament què estem tractant** en cada moment i escriure codi JavaScript que s'adapti al tipus de node adequat.

Informació bàsica que ens proporciona la interfície `Node`:

- **`nodeType`** → número que indica el tipus de node.

- **nodeName** → nom del node.

Valors més habituals de **nodeType**:

- 1 → **ELEMENT_NODE** (elements HTML com <div>, <p>...)
- 3 → **TEXT_NODE** (contingut textual)
- 8 → **COMMENT_NODE** (comentaris <!-- ... -->)
- 9 → **DOCUMENT_NODE** (document)

Per exemple:

```
1 <div>
2   <p id="primer">Primer paràgraf</p>
3 </div>
4
5 <script>
6   const primer = document.getElementById("primer");
7
8   console.log(document.nodeType); // 9 (DOCUMENT_NODE)
9   console.log(primer.nodeType);   // 1 (ELEMENT_NODE)
10
11   console.log(document.nodeName); // "#document"
12   console.log(primer.nodeName);   // "P"
13 </script>
```

1.1.6 Contingut textual: **textContent**, **innerText** i **innerHTML**

Els nodes del DOM disposen de diverses **propietats** per llegir i modificar el seu contingut textual. Les més utilitzades són:

- **textContent**
- **innerText**
- **innerHTML** (encara que aquesta última treballa també amb **etiquetes HTML**, no només amb text)

'textContent': text "pur" del node

Per llegir o escriure el **text d'un node i de tots els seus descendents** fem servir **textContent**. No interpreta etiquetes HTML, només tracta el contingut com a text:

```
1 <p id="missatge">Text original</p>
2
3 <script>
4   const missatge = document.getElementById("missatge");
```

```
5 // Llegir text
6 console.log(missatge.textContent); // "Text original"
7
8 // Escriure text missatge.textContent = "Text modificat";
9 </script>
```

- Si llegim `textContent`, obtenim **tot el text** del node i dels seus fills.
- Si assignem un valor a `textContent`, s'elimina el contingut anterior i es posa **només el nou text**.
- Com que no interpreta HTML, és més **segur** per evitar codi HTML o JavaScript inesperat (*injections*).

A diferència de `innerHTML`, `textContent` **no interpreta HTML**, només text.

Què és una injecció de codi?

Una **injecció de codi** és quan una aplicació tracta **dades** (text que ve d'un usuari o d'una font externa) com si fossin **codi executable** (o instruccions), i això permet que algú faci passar ordres que el sistema no havia previst (injecció).

Exemples típics:

1) Web: XSS (injecció a la pàgina): si una web posa contingut d'usuari amb ``innerHTML``, el navegador pot interpretar-ho com HTML/JS.

- Conseqüència: executar scripts al navegador de la víctima, robar dades, fer accions en nom seu...

2) Bases de dades: SQL Injection: si construïm una consulta SQL enganxant strings.

- Conseqüència: l'atacant pot alterar la consulta (llegir dades, modificar, esborrar,...).

3) Sistema: Command Injection: si es passa un text d'usuari a una comanda del sistema.

- Conseqüència: executar comandes al servidor.

'innerText': només el text visible

La propietat `innerText` també retorna text, però té en compte **l'estil i el rendement** de la pàgina:

- Només retorna el **text visible** (ignora el que està amagat amb `display: none`...).
- Pot ajustar els **salts de línia** segons com es mostri el contingut a la pantalla.
- Pot ser una mica **més lent** perquè necessita consultar el layout del navegador.

```
1 <p id="exemple">
2   Hola <span style="display: none">secret</span> món
3 </p>
```

```
4
5 <script>
6   const exemple = document.getElementById("exemple");
7   console.log(exemple.textContent); // "Hola secret món"   console.
8     log(exemple.innerText);    // "Hola món"
9 </script>
```

En aquest cas, `innerText` no inclou la paraula “secret” perquè el `span` està ocult visualment.

'innerHTML': text + etiquetes HTML

La propietat `innerHTML` permet treballar amb el **contingut HTML complet** d'un element (etiquetes + text):

- En llegir `innerHTML`, obtenim una **cadena amb el codi HTML intern** de l'element.
- En escriure a `innerHTML`, el navegador **torna a analitzar** aquesta cadena i crea nous nodes (elements, textos...) dins del DOM.

```
1 <p id="contingut"><strong>Hola</strong> món</p>
2
3 <script>
4   const contingut = document.getElementById("contingut");
5
6   console.log(contingut.innerHTML); // "<strong>Hola</strong> mó
7     n"
8
9   // Substituir tot el contingut per un altre HTML
10  contingut.innerHTML = "<em>Text en cursiva</em>";
11 </script>
```

Cal anar amb compte en **assignar valors a `innerHTML`** amb cadenes que vinguin de l'usuari (formularis, dades externes...), perquè es podria introduir **codi maliciós (XSS)**.

1.1.7 Compatibilitat entre navegadors (adaptacions)

En versions antigues dels navegadors, alguns mètodes/proprietats del DOM no estaven implementats o es comportaven de manera lleugerament diferent. Per això, en apunts antics es parla d'**adaptacions de codi per diferents navegadors** (polyfills, comprovacions de compatibilitat...). Tanmateix, **avui dia**:

- La major part del que utilitzarem (mètodes de `Document`, `Element` i `Node`) és **compatible** amb tots els navegadors moderns.

- Com que ECMAScript està en constant evolució és una bona pràctica comprovar si una funcionalitat existeix abans d'usar-la, sobretot quan es tracta de característiques molt noves.

1.2 Consulta i cerca del DOM

Un cop tenim clar que el DOM és un **arbre de nodes**, el següent pas és aprendre a **moure'ns per aquesta estructura i trobar els elements que necessitem**. Això implica tant desplaçar-nos per les relacions entre nodes (pares, fills, germans, descendents) com fer **cerques específiques** a partir de document o d'un element concret mitjançant diferents mètodes. En la documentació i en molts tutorials en anglès, aquest procés de recórrer l'arbre del DOM i seleccionar nodes s'anomena sovint **“traversing the DOM”**. En aquest apartat veurem quines propietats i mètodes ens ofereixen les interfícies Node, Document i Element per **navegar pel DOM i fer cerques eficients**.

1.2.1 Relacions entre nodes

La interfície Node inclou un conjunt de propietats que ens permeten **navegar per l'arbre del DOM** a partir de qualsevol node com a punt de referència. Gràcies a aquestes propietats podem accedir fàcilment al **pare**, als **fills**, als **germans anteriors i posteriors** o fins i tot al **primer i últim** fill d'un element. En combinació, aquestes referències ens permeten “pujar” i “baixar” per l'arbre, així com desplaçar-nos horitzontalment entre nodes del mateix nivell.

Algunes d'aquestes propietats són:

- parentNode → node pare.
- firstChild → primer fill.
- lastChild → últim fill.
- previousSibling → germà anterior.
- nextSibling → germà següent.

Per exemple:

```
1 <ul id="llista">
2   <li>Primer element</li>
3   <li id="central">Segon element</li>
4   <li>Tercer element</li>
5 </ul>
6
7 <script>
8   const central = document.getElementById("central");
```

```
9
10 console.log(central.parentNode);           // <ul id="
    llista">...</ul>
11 console.log(central.previousSibling);      // pot ser un node
    de text o <li> console.log(central.nextSibling); // igual: pot ser
    text o <li>
12 console.log(central.parentNode.firstChild); // primer fill (
    pot ser text)
13 console.log(central.parentNode.lastChild);  // últim fill
14 </script>
```

Les propietats `previousSibling` i `nextSibling` **treballen amb tots els tipus de nodes** (incloent-hi nodes de text creats per salts de línia o espais). Per treballar només amb elements veurem més endavant les propietats `previousElementSibling`, `nextElementSibling`, `firstElementChild` i `lastElementChild`.

1.2.2 Obtenció de descendents: `childNodes`, `children` i `hasChildNodes()`

Quan treballem amb el DOM sovint necessitem saber **quins nodes hi ha dins d'un altre node** (els seus fills directes). La interfície `Node` i, en el cas dels elements HTML, la interfície `Element`, ens ofereixen diverses propietats i mètodes per accedir-hi.

Les més habituals són:

- **`childNodes`** → col·lecció (`NodeList`) amb **tots** els fills del node: pot incloure **elements**, **nodes de text** (espais, salts de línia...) i altres tipus de nodes (comentaris...).
- **`children`** → col·lecció (`HTMLCollection`) amb **només els fills que són elements HTML**. No inclou ni textos ni comentaris. Aquesta propietat està disponible als **nodes de tipus `Element`** (no a qualsevol `Node`).
- **`hasChildNodes()`** → mètode que retorna `true` si el node té **com a mínim un fill** (de qualsevol tipus) i `false` en cas contrari.

Tant `childNodes` com `children` retornen col·leccions **semblants a un array** que podem recórrer amb `for...of`.

Exemple: diferència entre `childNodes` i `children`

```
1 <ul id="llista">
2   <li>Primer element</li>
3   <!-- Un comentari -->
4   <li>Segon element</li>
5 </ul>
6
7 <script>
8   const llista = document.getElementById("llista");
9
```

```
10 console.log("Té fills?", llista.hasChildNodes()); // true
11
12 console.log("=== childNodes ==="); for (const node of llista.childNodes)
13 {
14     console.log(node.nodeName, node.nodeType);
15 }
16 console.log("=== children ===");
17 for (const element of llista.children) {
18     console.log(element.nodeName, element.textContent);
19 }
20 </script>
```

En aquest exemple:

- `llista.childNodes` pot contenir, per exemple:
 - #text (espais/salts de línia abans del primer `<li>`)
 - LI
 - #text (salts de línia entre elements)
 - #comment
 - LI
 - #text (salts de línia finals)
- `llista.children`, en canvi, **només** conté:
 - LI → "Primer element"
 - LI → "Segon element"

Això fa que `children` sigui molt pràctic quan només volem **treballar amb els elements HTML** i ignorar els nodes de text creats pels espais i salts de línia del codi.

1.2.3 Cerca simple d'elements al document

La interfície Document és el punt d'entrada principal per **cercar elements** en el DOM. Entre els mètodes més utilitzats tenim:

- **`document.getElementById(id)`**: Retorna **un únic element** amb l'atribut `id` indicat (o null si no existeix).
- **`document.getElementsByTagName(tag)`**: Retorna una col·lecció (HTMLCollection) amb **tots els elements** d'un tipus determinat ("`p`", "`li`", "`div`"...).
- **`document.getElementsByClassName(className)`**: Retorna tots els elements que contenen una classe concreta.

Per exemple:

```
1 <h1 id="titol-principal">Títol</h1>
2 <p class="resum">Resum 1</p>
3 <p class="resum">Resum 2</p>
4
5 <script>
6     const titol = document.getElementById("titol-principal");
7     console.log(titol.textContent);
```

```
8
9  const paragraphs = document.getElementsByTagName("p");
10 console.log("Nombre de paràgrafs:", paragraphs.length);
11
12 const resums = document.getElementsByClassName("resum"); for (
13     const p of resums) {
14     console.log(p.textContent);
15 }
</script>
```

Col·leccions “vives”: les col·leccions retornades per `getElementsByTagName` i `getElementsByClassName` s'actualitzen automàticament si s'afegeixen o s'eliminen elements del document.

1.2.4 Cerca amb selectors CSS: `querySelector` i `querySelectorAll`

Els mètodes `querySelector` i `querySelectorAll` permeten fer cerques més **flexibles** gràcies als **selectors CSS**.

- **`document.querySelector(selector)`:** retorna **el primer** element que compleix el selector.
- **`document.querySelectorAll(selector)`:** retorna **tots** els elements que compleixen el selector (`NodeList`).

Selectors habituals:

- `#identificador` → `id="identificador"`
- `.nom_classe` → `class="nom_classe"`
- `element` → etiqueta (`h1`, `p`, `li`, ...)
- `element1 element2` → `element2` descendent (no cal que sigui directe) d'`element1`
- `element1 > element2` → `element2` fill directe d'`element1`
- `[type="button"]` → elements amb atribut `type="button"`
- `li:first-child` → primer fill de la seva llista, etc.

Exemple d'ús de '`querySelectorAll`'

```
1 <h1>Primera capçalera</h1>
2 <ul>
3   <li>Primer element de la primera llista</li>
4   <li>Segon element de la primera llista</li>
5   <li>Tercer element de la primera llista</li>
6 </ul>
7
8 <h1 class="secundari">Segona capçalera</h1>
```

```
9 <p id="contingut" class="secundari paragraf">Un paràgraf</p>
10
11 <ul>
12   <li>Primer element de la segona llista</li>
13   <li>Segon element de la segona llista</li>
14   <li>Tercer element de la segona llista</li></ul>
15
16 <script>
17   console.log("-- Capçaleres h1 amb classe 'secundari' --");
18   const titols = document.querySelectorAll("h1.secundari");
19   for (const element of titols) {
20     console.log(element.textContent);
21   }
22
23   console.log("-- Tots els elements de llista (li) --");
24   const totsLi = document.querySelectorAll("li");
25   for (const li of totsLi) {
26     console.log(li.textContent);
27   }
28
29   console.log("-- Primers elements de cada llista --");
30   const primers = document.querySelectorAll("li:first-child");
31   for (const li of primers) {
32     console.log(li.textContent);
33   }
34 </script>
```

En l'últim cas d'aquest exemple, només se seleccionen els **primers elements** de cada llista, gràcies al selector `li:first-child`.

1.2.5 Cerca relativa a un element: mètodes de 'Element'

A més de Document, també podem cercar **a partir d'un element concret**. Això és molt útil per **limitar l'àrea de cerca** als descendents d'un node.

Per exemple, la interfície Element disposa de:

- `element.getElementsByTagName(tag)`
- `element.getElementsByClassName(className)`
- `element.querySelector(selector)`
- `element.querySelectorAll(selector)`

Aquests mètodes funcionen igual que els de Document, però només busquen **dins dels descendents** d'aquell element.

Exemple amb '' i subllista

```
1 <ul>
2   <li>Primer element de la llista</li>
3   <li id="central">
4     <ul>
5       <li>Primer element de la subllista</li>
6       <li>Segon element de la subllista</li>
7     </ul>
8   </li>
9   <li>Tercer element de la llista</li>
10 </ul>
11
12 <script>
13   const central = document.getElementById("central");
14   const subElements = central.getElementsByTagName("li");
15
16   for (const li of subElements) {
17     console.log(li.textContent);
```

```
18 }  
19 </script>
```

En aquest exemple:

- Primer fem servir `document.getElementById("central")` per obtenir l'element amb id "central".
- Després fem la cerca de `<li>` **a partir d'aquest element**, no de `document`, de manera que només obtenim els elements de la **subllista**.

1.2.6 Bones pràctiques de consulta i cerca del DOM

Algunes recomanacions per treballar de manera eficient:

- **Fer servir id quan es tingui un element únic:** és més clar i ràpid utilitzar `document.getElementById("...")`.
- **Limitar l'àrea de cerca:** si només interessa una secció concreta de la pàgina, caldria obtenir primer l'element contenidor i buscar la resta a partir d'ell (`contenidor.querySelectorAll(...)`).
- **Reutilitzar resultats:** si es necessita la mateixa col·lecció diverses vegades, una bona opció és desar-la en una variable en comptes de tornar-la a calcular.
- **Ús de selectors CSS per cerques complexes:** per combinacions de classes, atributs, jerarquies..., `querySelector` i `querySelectorAll` són molt expressius i flexibles.

2. Programació amb el DOM (II)

En aquesta segona lliçó de DOM, aprofundirem en tot allò que ja hem començat a practicar: aprendrem a actualitzar de manera sistemàtica el contingut i els atributs dels elements, a crear, clonar, eliminar i reemplaçar nodes, a modificar-ne els estils amb classes i style, i a connectar-ho amb la detecció d'esdeveniments perquè cada acció de l'usuari pugui transformar directament l'estructura i l'aparença de la pàgina.

2.1 Actualització dels elements del DOM

Un cop sabem localitzar els elements del document amb mètodes com `getElementById`, `querySelector` o `querySelectorAll`, el següent pas és aprendre **què podem** fer amb aquests elements un cop els tenim referenciats. En aquest apartat veurem com **actualitzar el contingut i les dades dels nodes del DOM**: canviarem el text que mostren, modificarem i consultarem atributs, afegirem o eliminarem elements de l'arbre i ajustarem la seva aparença. En tots els exemples partirem sempre d'una referència prèvia a l'element que volem modificar, obtinguda amb algun dels mètodes de cerca que ja coneixem.

2.1.1 Modificar el contingut dels elements

En la lliçó anterior (*Programació amb el DOM I*) ja vam veure les propietats `textContent`, `innerText` i `innerHTML` per **llegir o “seleccionar” el text** dels elements (per exemple, per mostrar-lo amb `console.log(titol.textContent)`). Ara farem un pas més: les farem servir sobretot per **canviar o substituir el contingut** que es mostra a la pàgina.

Suposarem que ja tenim les referències als elements que volem modificar:

```
1 const titol = document.getElementById("titol");  
2 const paragraf = document.querySelector("p");
```

textContent: canviar només el text (opció segura)

`textContent` permet **substituir tot el text** d'un element (i dels seus descendents) sense interpretar HTML:

```
1 paragraf.textContent = "Aquest és un nou text sense cap etiqueta  
HTML.";
```

Característiques:

- Elimina el text anterior i el reemplaça pel nou.
- **No interpreta etiquetes HTML** (, <i>...): es mostren com a text literal.
- És l'opció **recomanada** quan només vols canviar text i evitar possibles injeccions de codi.

innerHTML: canviar el contingut amb HTML

innerHTML permet substituir tot el contingut intern d'un element, incloent-hi **etiquetes HTML i fills**:

```
1 titol.innerHTML = 'Nou <strong>títol</strong> amb destacat';  
2 paragraf.innerHTML = 'Text amb un <a href="#">enllaç</a>.';
```

Característiques:

- El navegador interpreta la cadena com a **codi HTML** i crea de nou els nodes interns.
- És útil quan vols generar contingut amb **format** (negretes, enllaços, llistes...).

Atenció amb dades d'usuari (XSS): si es construeix la cadena d'innerHTML a partir de dades que venen d'un formulari, d'un servei extern..., i no es validen, es pot introduir codi HTML/JavaScript maliciós (injeccions XSS).

Per això, quan només cal text, és millor fer servir `textContent`.

innerText: text visible segons l'estil

innerText també permet llegir i escriure text, però té en compte el **layout i l'estil** de la pàgina (només el text visible):

```
1 paragraf.innerText = "Text visible per a l'usuari.";
```

Característiques:

- Té en compte coses com `display: none` o certs canvis de línia.
- El seu comportament pot variar una mica entre navegadors.

Per a la majoria d'exemples i activitats ens centrarem sobretot en:

- `textContent` → quan vulguem **canviar text pla** de forma segura.
- `innerHTML` → quan necessitem **substituir el contingut amb HTML** (etiquetes).

2.1.2 Modificar atributs

A més del text, els elements HTML també tenen **atributs** (id, href, title, src...) que vam aprendre a **llegir** en la lliçó anterior mitjançant propietats i mètodes de la interfície Element. En aquest apartat veurem com, un cop tenim l'element localitzat (per exemple amb getElementById o querySelector), podem **consultar i modificar aquests atributs des de JavaScript**: llegirem el seu valor actual, n'assignarem de nous, en canviarem el contingut i, si cal, els eliminarem. Això ens permetrà adaptar en temps real enllaços, imatges, camps de formulari i qualsevol altre component de la pàgina.

Mètodes genèrics de la interfície Element:

```
1 const enllac = document.querySelector("a");
2
3 // Llegir un atribut
4 const url = enllac.getAttribute("href");
5
6 // Assignar o modificar un atribut
7 enllac.setAttribute("href", "https://www.ioc.cat");
8
9 // Eliminar un atribut
10 enllac.removeAttribute("title");
```

A més, molts d'aquests atributs també es poden gestionar com a **propietats “especials”** de l'objecte Element en JavaScript. El navegador crea propietats directes per als atributs més habituals (id, value, disabled, checked...), de manera que podem llegir i modificar el seu valor amb la sintaxi de punt (element.propietat) sense haver de cridar sempre getAttribute() o setAttribute().

Aquesta sintaxi sol ser més còmoda quan treballem amb formularis i elements interactius:

```
1 const input = document.querySelector("input");
2
3 // Equivalents sovint a atributs
4 input.id = "usuari";           // semblant a setAttribute("id", "usuari")
5 input.value = "Anna";         // valor del camp de formulari
6 input.disabled = true;        // desactiva el camp
7 input.checked = true;         // per a <input type="checkbox">
```

Atributs de dades (data-*)

HTML5 permet definir **atributs personalitzats** en qualsevol etiqueta fent servir el prefix data-. Aquests atributs (per exemple data-id, data-rol, data-preu...) no tenen cap efecte visual per si mateixos, però serveixen per **emmagatzemar informació addicional** relacionada amb l'element que després podrem llegir o modificar des de JavaScript amb mètodes com getAttribute("data-id") o mitjançant la propietat dataset:

```
1 <li data-preu="12.5">Producte X</li>
```

A JavaScript hi accedim via dataset:

```
1 const item = document.querySelector("li");
2 const preu = parseFloat(item.dataset.preu); // "12.5" → 12.5
3 item.dataset.preu = "15.0";
```

2.1.3 Crear i inserir nous elements

Fins ara hem treballat amb elements que **ja existien al document HTML**: els hem localitzat, n'hem canviat el text i els atributs, i n'hem modificat l'aparença. En aquest apartat anirem un pas més enllà i aprendrem a **crear nous nodes des de JavaScript** (per exemple nous paràgrafs, elements de llista o targetes d'un bloc) i a **inserir-los dins del DOM** en la posició que ens interessi. Per fer-ho utilitzarem mètodes com `createElement`, `createTextNode`, `appendChild` o `append`, sempre partint d'un element pare que ja tinguem seleccionat amb algun dels mètodes de cerca que coneixem.

Per **afegir contingut nou** al DOM seguim normalment tres passos:

1. **Crear** el node.
2. (Opcional) **Configurar-lo** (text, atributs, classes...).
3. **Inserir-lo** en un punt concret de l'arbre.

`document.createElement()` i `document.createTextNode()`:

```
1 const llista = document.getElementById("llista");
2
3 // 1) Crear elements
4 const nouLi = document.createElement("li");
5 const text = document.createTextNode("Element afegit dinàmicament");
6
7 // 2) Afegir contingut al <li>
8 nouLi.appendChild(text);
9
10 // 3) Inserir el <li> a la llista
11 llista.appendChild(nouLi);
```

També podem assignar el text directament:

```
1 const nouLi2 = document.createElement("li");
2 nouLi2.textContent = "Un altre element";
3 llista.appendChild(nouLi2);
```

Inserir abans d'un altre element: `insertBefore`:

```
1 const primer = llista.firstElementChild;
2 llista.insertBefore(nouLi, primer); // insereix abans del
  primer element
```

En versions modernes del DOM també tenim mètodes com `before`, `after`, `append` i `prepend` sobre els propis elements:

```
1 llista.prepend(nouLi); // insereix com a primer fill
2 llista.append(nouLi2); // insereix com a últim fill
```

2.1.4 Clonar elements

En lloc de crear cada element nou des de zero amb `createElement` i `appendChild`, el DOM ens permet **duplicar un node ja existent** mitjançant el mètode `cloneNode()`. Això és especialment útil quan volem repetir una mateixa estructura (per exemple, una targeta de producte, una fila d'una taula, un element de llista amb diversos subelements...) canviant només alguna part del contingut:

```
1 const original = document.querySelector(".targeta");
2 const copia = original.cloneNode(true); // clonem la targeta
```

`cloneNode(true)` vs `cloneNode(false)`

El mètode `cloneNode()` rep un paràmetre booleà que indica si volem copiar també els fills:

```
1 const original = document.querySelector(".element");
2
3 // Clon superficial (sense fills)
4 const clonSimple = original.cloneNode(false);
5
6 // Clon profund (amb tots els fills)
7 const clonCompleto = original.cloneNode(true);
```

- `cloneNode(false)`
 - Crea una còpia **només de l'element** (mateixa etiqueta, mateixos atributs).
 - **No** inclou els nodes fills (ni textos ni altres elements).
- `cloneNode(true)`
 - Crea una còpia de l'element **i de tot el seu subarbre** (fills, nets, text...).
 - És el més habitual quan volem duplicar una estructura completa.

Inserir el clon al DOM

Recordem que `cloneNode()` només crea el nou node en memòria. Perquè aparegui a la pàgina, l'hem d'**inserir** com qualsevol altre element:

```
1 const llista = document.getElementById("llista");
2 const original = document.querySelector(".element-llista");
3
4 // Clonem l'element amb tot el seu contingut
5 const clon = original.cloneNode(true);
6
7 // (Opcional) Modificar alguna part del clon
8 clon.textContent = "Element duplicat";
9
10 // Inserir el clon al final de la llista
11 llista.appendChild(clon);
```

Podem aprofitar el clon com a **plantilla** i, abans d'afegir-lo al DOM, canviar-ne el text, els atributs o les classes.

Compte amb els identificadors (id)

En clonar elements cal vigilar amb l'atribut `id`:

```
1 <li id="element-exemple">Text original</li>
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111
112
113
114
115
116
117
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
```

Ara tindríem **dos elements amb el mateix id**, cosa que **no és recomanable** (l'atribut `id` ha de ser únic a la pàgina). En aquests casos, és millor:

- Eliminar o modificar l'`id` del clon:

```
1 clon.removeAttribute("id");
2 // o bé
3 clon.id = "element-exemple-2";
```

- O bé fer servir classes (`class`) en lloc d'`id` quan sabem que hi haurà múltiples còpies.

Ús típic com a plantilla

Un patró molt habitual és tenir una **plantilla oculta** al HTML i clonar-la cada vegada que necessitem un nou element:

```
1 <ul id="llista"></ul>
2
3 <li id="plantilla" class="element" style="display: none;">
4   Element de la llista
5 </li>
```

```
1 const llista = document.getElementById("llista");
2 const plantilla = document.getElementById("plantilla");
3
4 function afegirElement(text) {
5     const clon = plantilla.cloneNode(true);
6     clon.removeAttribute("id");           // evitem duplicar l'id
7     clon.style.display = "";             // el fem visible
8     clon.textContent = text;             // personalitzem el
        contingut
9     llista.appendChild(clon);
10 }
11
12 afegirElement("Primer element");
13 afegirElement("Segon element");
```

D'aquesta manera, aprofitem l'HTML com a base i, amb `cloneNode(true)`, podem generar tantes còpies com calgui sense redefinir tota l'estructura des de JavaScript.

2.1.5 Eliminar i reemplaçar elements

Igual que podem afegir nous nodes al document, sovint també necessitarem **treure elements que ja no ens interessin** o bé **substituir-los per uns de nous** (per exemple, esborrar un element d'una llista, eliminar un missatge d'error o canviar una targeta sencera per una altra). En aquest apartat veurem com, a partir d'un element que tinguem seleccionat, podem **eliminar-lo del DOM** amb mètodes com `remove()` o `removeChild()`, i com podem reemplaçar un node per un altre fent servir `replaceChild()` o combinant eliminació i inserció. Això ens permetrà mantenir el contingut de la pàgina sempre actualitzat segons el que passi a l'aplicació.

Eliminar un node:

Si tenim una referència al node i al seu pare:

```
1 const aEliminar = document.querySelector(".element-eliminar");
2 aEliminar.parentNode.removeChild(aEliminar);
```

En navegadors moderns, podem fer-ho més directe:

```
1 aEliminar.remove();
```

Buidar el contingut d'un element:

```
1 const contenidor = document.getElementById("missatges");
2 contenidor.innerHTML = ""; // elimina tots els fills de cop
```

Alternativament (mètode clàssic):

```
1 while (contenidor.firstChild) {
```

```
2   contenidor.removeChild(contenidor.firstChild);  
3 }
```

Reemplaçar un node:

```
1 const vell = document.getElementById("banner-vell");  
2 const nou = document.createElement("div");  
3 nou.textContent = "Nou banner";  
4  
5 vell.parentNode.replaceChild(nou, vell);
```

2.2 Modificació d'estils CSS

Fins ara hem definit l'aparença de la pàgina principalment amb fulls d'estil CSS aplicats a partir de classes i selectors al fitxer HTML. Un cop disposem de referències als elements del DOM, però, també podem canviar els seus estils des de JavaScript: afegint o traient classes, modificant propietats concretes (color, mides, marges...) o aplicant estils temporals en resposta a una acció de l'usuari. En aquest apartat veurem com utilitzar sobretot la propietat `classList` per activar i desactivar classes CSS (la forma més recomanable) i, en casos puntuals, la propietat `style` per ajustar valors concrets directament des del codi.

2.2.1 Propietat style

La manera més directa de canviar l'aparença d'un element des de JavaScript és utilitzar la propietat `style` de l'objecte `Element`. Aquesta propietat ens permet accedir i modificar els **estils en línia** de l'element, és a dir, els mateixos valors que escriuríem a l'atribut `style` de l'HTML (`<p style="color: red;">...</p>`). D'aquesta manera podem canviar, des del codi, propietats com el color del text, el fons, l'amplada, l'alçada, els marges...

Tot i que és una eina molt útil per fer **ajustos puntuals** (per exemple, amagar o mostrar un element, remarcar temporalment un error, moure lleugerament un bloc...), en aplicacions més grans intentarem continuar utilitzant sobretot **classes CSS** per mantenir l'estil separat de la lògica. En aquest apartat veurem com llegir i modificar propietats bàsiques amb `element.style`, i quines convencions hem de seguir per escriure noms de propietats CSS en JavaScript:

```
1 const bloc = document.getElementById("bloc");  
2  
3 bloc.style.backgroundColor = "lightblue";  
4 bloc.style.border = "1px solid black";  
5 bloc.style.padding = "10px";
```

Recordem:

- Les propietats amb guions (`background-color`) es passen a **camelCase** (`backgroundColor`).
- Aquestes modificacions només afecten els **estils en línia**; tot allò que està definit als fulls CSS, continua aplicant-se i pot prevaldre segons l'especificitat.

Per llegir el valor **real** d'un estil (tenint en compte fulls CSS, herència...), podem fer servir `window.getComputedStyle(element)`:

```
1 const estilCalculat = window.getComputedStyle(bloc);  
2 console.log(estilCalculat.backgroundColor);
```

2.2.2 Treballar amb classes: `className` i `classList`

En la majoria dels casos, l'aparença d'un element ve definida per les **classes CSS** que té assignades (`class="targeta destacat error"...`). Des de JavaScript podem llegir i modificar aquestes classes per **activar o desactivar estils** en funció del que passi a la nostra aplicació (per exemple, marcar un camp com a incorrecte, amagar/mostrar un menú, aplicar un "mode fosc"...).

Els dos punts principals d'accés a les classes d'un element són:

- la propietat **`className`**, que conté **totes les classes en una sola cadena** de text;
- la propietat **`classList`**, que proporciona una col·lecció de classes amb mètodes per **afegir, eliminar o alternar** valors de forma més còmoda.

`className`: totes les classes en una cadena

`className` és una propietat que guarda el contingut de l'atribut `class` com una string:

```
1 <p id="missatge" class="info destacat">Text...</p>
```

```
1 const missatge = document.getElementById("missatge");  
2  
3 console.log(missatge.className); // "info destacat"  
4  
5 // Substituïm totes les classes per una de nova  
6 missatge.className = "error";
```

Característiques de `className`:

- Quan es **llegeix** `element.className` s'obté una cadena amb totes les classes separades per espais.

- Quan **s'assigna** (`element.className = "nova-classe"`) s'està **substituint tota la llista de classes** anterior.
- És útil si es vol **establir de cop** totes les classes d'un element (per exemple, quan tens plantilles molt controlades).

El problema és que és fàcil **sobreescriure classes que no es volien tocar**. Per exemple:

```
1 // L'element tenia "info destacat"
2 missatge.className = "error";
3 // → Ara ha perdut "info" i "destacat"
```

Per evitar això, és molt millor treballar amb `classList`.

classList: afegir, treure i alternar classes

`classList` és una propietat que retorna un objecte especial (`DOMTokenList`) amb totes les classes de l'element i una sèrie de mètodes molt útils:

```
1 const missatge = document.getElementById("missatge");
2
3 // Afegeix una classe si no hi és
4 missatge.classList.add("error");
5
6 // Elimina una classe si hi és
7 missatge.classList.remove("info");
8
9 // Alterna una classe (si hi és la treu, si no hi és la posa)
10 missatge.classList.toggle("actiu");
11
12 // Comprova si l'element té una classe concreta
13 if (missatge.classList.contains("error")) {
14     console.log("Hi ha un error!");
15 }
```

Mètodes principals de `classList`:

- **add("classe")** – afegeix una o més classes sense esborrar les que ja hi ha.
- **remove("classe")** – elimina una o més classes si estan presents.
- **toggle("classe")** – si la classe **no hi és**, l'afegeix; si **ja hi és**, l'elimina.
- **contains("classe")** – retorna `true` si l'element té aquella classe, `false` en cas contrari.
- **replace("antic", "nou")** – substitueix una classe per una altra (no tots els navegadors antics la suporten, però actualment és habitual).

A diferència de `className`, `classList` **no sobrescriu tota la cadena** de classes, sinó que treballa de forma incremental, cosa que el fa molt més segur i recomanable per a aplicacions reals.

Exemple: alternar una classe de "mode fosc"

Suposem que tenim un botó per activar i desactivar un **mode fosc** afegint o traient una classe al <body>:

```
1 <button id="boto-mode">Canvia mode</button>

1 /* Estil per al mode fosc */
2 .mode-fosc {
3     background-color: #222;
4     color: #f5f5f5;
5 }

1 const boto = document.getElementById("boto-mode");
2 const cos = document.body;
3
4 boto.addEventListener("click", function () {
5     cos.classList.toggle("mode-fosc");
6 });
```

Cada vegada que fem clic al botó:

- si body **no** té la classe mode-fosc, `classList.toggle("mode-fosc")` l'afegeix → s'aplica el mode fosc;
- si body **ja** té la classe, `toggle` la treu → tornem al mode clar.

2.2.3 Exemple: remarcar errors en un formulari

En aquest apartat aplicarem el que hem vist sobre **classList i modificació d'estils** per resoldre una situació molt habitual: remarcar visualment els **errors d'un formulari**. A partir d'uns quants camps senzills (nom, correu, missatge...), farem una validació bàsica amb JavaScript i, en funció del resultat, afegirem o traurem classes CSS per marcar els camps incorrectes (canviant-ne el color, el fons, mostrant missatges d'error...). D'aquesta manera veurem com la manipulació del DOM ens permet adaptar tant el contingut com l'aparença de la pàgina en resposta a les accions de l'usuari.

CSS associat (separat al full d'estils):

```
1 <!DOCTYPE html>
2 <html lang="ca">
3 <head>
4     <meta charset="utf-8">
5     <title>Exemple DOM - Remarcar errors en un formulari</title>
6     <style>
7         body {
8             font-family: system-ui, sans-serif;
9             max-width: 600px;
10            margin: 2rem auto;
11            padding: 0 1rem;
12            line-height: 1.5;
13        }
14
15        h1 {
16            text-align: center;
17            margin-bottom: 1.5rem;
18        }
19    </style>
```

```
20     form {
21         border: 1px solid #ccc;
22         border-radius: 8px;
23     padding: 1.5rem;         background-color: #f9f9f9;
24     }
25
26     .grup-camp {
27         margin-bottom: 1rem;
28     }
29
30     label {
31         display: block;
32         font-weight: 600;
33         margin-bottom: 0.25rem;
34     }
35
36     input[type="text"],
37     input[type="email"],
38     textarea {
39         width: 100%;
40         padding: 0.5rem;
41         border: 1px solid #ccc;
42         border-radius: 4px;
43         box-sizing: border-box;
44         font: inherit;
45     }
46
47     textarea {
48         min-height: 100px;
49         resize: vertical;
50     }
51
52     .camp-error {
53         border-color: red;
54         background-color: #ffe6e6;
55     }
56
57     .missatge-error {
58         color: red;
59         font-size: 0.9rem;
60         margin-top: 0.25rem;
61         display: none; /* per defecte ocult */
62     }
63
64     .missatge-error.visible {
65         display: block;
66     }
67
68     .missatge-ok {
69         margin-top: 1rem;
70         padding: 0.75rem;
71         border-radius: 4px;
72         background-color: #e2f7e2;
73         color: #165c1b;
74         border: 1px solid #9ad19a;
75         display: none; /* es mostrarà només quan tot vagi bé */
76     }
77
78     button[type="submit"] {
79         background-color: #0069d9;
80         color: #fff;
81         border: none;
82         border-radius: 4px;
```

```
83     padding: 0.6rem 1.2rem;
84 font: inherit;      cursor: pointer;
85   }
86
87   button[type="submit"]:hover {
88     background-color: #0052aa;
89   }
90 </style>
91 </head>
92 <body>
93
94   <h1>Formulari de contacte</h1>
95
96   <form id="formulari-contacte" novalidate>
97     <div class="grup-camp">
98       <label for="nom">Nom complet</label>
99       <input type="text" id="nom" name="nom">
100       <p class="missatge-error" id="error-nom">
101         Introdueix el teu nom (mínim 2 caràcters).
102       </p>
103     </div>
104
105     <div class="grup-camp">
106       <label for="email">Correu electrònic</label>
107       <input type="email" id="email" name="email">
108       <p class="missatge-error" id="error-email">
109         Introdueix un correu electrònic vàlid.
110       </p>
111     </div>
112
113     <div class="grup-camp">
114       <label for="missatge">Missatge</label>
115       <textarea id="missatge" name="missatge"></textarea>
116       <p class="missatge-error" id="error-missatge">
117         El missatge ha de tenir com a mínim 10 caràcters.
118       </p>
119     </div>
120
121     <button type="submit">Enviar</button>
122
123     <div class="missatge-ok" id="missatge-ok">
124       Formulari enviat correctament. Gràcies pel teu missatge!
125     </div>
126   </form>
127
128   <script>
129     const formulari = document.getElementById("formulari-
130       contacte");
131     const campNom = document.getElementById("nom");
132     const campEmail = document.getElementById("email");
133     const campMissatge = document.getElementById("missatge");
134
135     const errorNom = document.getElementById("error-nom");
136     const errorEmail = document.getElementById("error-email");
137     const errorMissatge = document.getElementById("error-
138       missatge");
139
140     const missatgeOk = document.getElementById("missatge-ok");
141
142     formulari.addEventListener("submit", function (event) {
143       event.preventDefault(); // Evitem l'enviament real del
144       formulari
```

```
143 // Eliminem estats d'error anteriors
144 campNom.classList.remove("camp-error");      campEmail.classList
      .remove("camp-error");
145 campMissatge.classList.remove("camp-error");
146
147 errorNom.classList.remove("visible");
148 errorEmail.classList.remove("visible");
149 errorMissatge.classList.remove("visible");
150
151 missatgeOk.style.display = "none";
152
153 let hiHaErrors = false;
154
155 // Validació del nom
156 const valorNom = campNom.value.trim();
157 if (valorNom.length < 2) {
158     hiHaErrors = true;
159     campNom.classList.add("camp-error");
160     errorNom.classList.add("visible");
161 }
162
163 // Validació bàsica del correu (comprovem que contingui "@"
    ")
164 const valorEmail = campEmail.value.trim();
165 if (valorEmail === "" || !valorEmail.includes("@")) {
166     hiHaErrors = true;
167     campEmail.classList.add("camp-error");
168     errorEmail.classList.add("visible");
169 }
170
171 // Validació del missatge
172 const valorMissatge = campMissatge.value.trim();
173 if (valorMissatge.length < 10) {
174     hiHaErrors = true;
175     campMissatge.classList.add("camp-error");
176     errorMissatge.classList.add("visible");
177 }
178
179 // Si no hi ha errors, mostrem un missatge de confirmació
180 if (!hiHaErrors) {
181     missatgeOk.style.display = "block";
182     formulari.reset(); // Buidem els camps
183 }
184 });
185 </script>
186
187 </body>
188 </html>
```

2.3 Afegir i eliminar la detecció d'events

Trobareu la lliçó 2, "Programació amb esdeveniments", a la unitat 2.

Anteriorment, quan veu estudiar la **programació amb esdeveniments**, ja vaureu aprendre què és un esdeveniment, com funciona el patró `addEventListener(...)` i com podem **recollir la informació** que ens arriba a través de l'objecte event (e.target, e.key, e.preventDefault(...)).

En aquell moment, però, tot just començàvem a treballar amb el DOM i encara teníem **pocs elements “reals” sobre els quals actuar**: ens limitàvem sobretot a mostrar missatges, escriure per consola o canviar algun text molt senzill.

Ara, en canvi, disposem de tot el que hem après en aquesta unitat de DOM: sabem **localitzar elements, canviar-ne el contingut, modificar atributs, afegir i eliminar nodes, treballar amb classes i estils...** Això ens permet que la detecció d'esdeveniments tingui molta més utilitat: cada clic, cada tecla o cada enviament d'un formulari pot **desencadenar modificacions al mateix DOM** (afegir una targeta nova, remarcar un error, mostrar o amagar una secció, clonar elements, actualitzar una llista...).

En aquest apartat revisarem com **afegir i eliminar gestors d'esdeveniments** sobre elements que ja tenim seleccionats del DOM:

- Recordarem l'ús de `addEventListener()` per **fer que un element “reaccioni”** a un esdeveniment (per exemple, un botó que afegix un nou `<li>` a una llista, o un enllaç que mostra/oculta un bloc de text).
- Veurem com utilitzar `removeEventListener()` per **desactivar aquesta reacció** quan ja no ens interessi (per exemple, desconnectar temporalment un botó, o deixar d'escoltar els clics d'una zona concreta).
- Posarem especial atenció al fet que, per poder eliminar un *listener*, hem de conservar una **referència a la funció** que vam passar a `addEventListener`, ja que `removeEventListener` necessita exactament el mateix tipus d'esdeveniment i la mateixa funció.

L'objectiu és connectar el que ja sabem sobre esdeveniments amb totes les tècniques de manipulació del DOM que hem treballat fins ara, de manera que els **elements HTML deixin de ser estàtics** i passin a **respondre activament a les accions de l'usuari**.

2.3.1 Afegir detectors amb `addEventListener`

A la lliçó de **Programació amb esdeveniments** ja vam veure que la manera moderna i recomanada d'assignar gestors d'esdeveniments és fer servir el mètode `addEventListener(...)`, i el vam utilitzar per mostrar missatges, escriure per consola o actualitzar algun text senzill.

Ara, però, ja sabem **localitzar i modificar elements del DOM**, crear i inserir nous nodes, canviar classes... Això vol dir que cada vegada que es dispara un esdeveniment podem aprofitar el gestor per **tocar directament l'estructura de la pàgina**.

Suposem que tenim un botó per afegir elements a una llista:

```
1 <button id="afegir">Afegir element</button>
2 <ul id="llista"></ul>
```

Des de JavaScript:

```
1 // 1. Seleccióem els elements del DOM amb què treballarem
2 const botoAfehir = document.getElementById("afehir");
3 const llista = document.getElementById("llista");
4 let comptador = 0;
5
6 // 2. Definim la funció que s'executarà quan es faci clic al botó
7 function afehirElement() {
8     comptador++;
9
10    // Creem un nou <li> i en modifiquem el contingut
11    const nouElement = document.createElement("li");
12    nouElement.textContent = "Element " + comptador;
13
14    // Inserim el nou node al DOM
15    llista.appendChild(nouElement);
16 }
17
18 // 3. Afehim el gestor d'esdeveniments al botó
19 botoAfehir.addEventListener("click", afehirElement);
```

Cada cop que l'usuari fa clic al botó:

1. Es dispara l'esdeveniment click sobre el node botoAfehir.
2. El navegador crida la funció afehirElement.
3. Dins la funció, **creem un nou node i l'afegim al DOM**, de manera que la pàgina canvia en temps real.

Recordatori de l'estructura 'addEventListener'

El mètode addEventListener té aquesta forma general:

```
1 element.addEventListener(tipus, callback, opcions);
```

- **tipus**: cadena amb el nom de l'esdeveniment ("click", "submit", "keyup", "change", "input"...).
- **callback**: funció que s'executarà quan es dispari l'esdeveniment. Rep com a primer paràmetre un objecte Event amb informació del que ha passat.
- **opcions** (opcional): objecte amb configuracions addicionals ({ once: true }, capture: true...), que ja vam treballar als apunts d'esdeveniments.

Com que ara ja tenim el DOM "a les nostres mans", el patró bàsic que seguirem en molts exercicis serà:

1. **Seleccionar** els elements del DOM que ens interessin.
2. **Afehir-hi** un o més gestors amb addEventListener(...).

3. Dins del gestor, **llegir l'objecte event si cal** (`event.target`, `event.preventDefault(...)`) i **modificar el DOM**: canviar text, atributs, classes, crear o eliminar nodes...

2.3.2 Eliminar detectors amb `removeEventListener`

Igual que podem **afegir** gestors d'esdeveniments a un element del DOM amb `addEventListener`, també els podem **eliminar** quan ja no ens interessa continuar escoltant aquell esdeveniment. Això és útil, per exemple, quan un botó només ha de funcionar **durant un temps limitat**, quan un joc s'ha acabat, quan volem "bloquejar" una acció després d'usar-la un cop o quan estem netejant codi abans d'eliminar un element del DOM.

La forma general és:

```
1 element.removeEventListener(tipus, callback, opcions);
```

- **tipus**: el mateix nom de l'esdeveniment que vam fer servir a `addEventListener` ("click", "keydown", "submit" ...).
- **callback**: exactament la mateixa funció que vam passar a `addEventListener`.
- **opcions** (si s'han fet servir): han de coincidir també (per exemple, si es va usar `capture: true`).

Exemple: desactivar un botó després de fer-lo servir

Suposem que tenim un botó que només ha de funcionar **una vegada**: afegeix un element i després deixa de respondre als clics:

```
1 <button id="un-cop">Fes-ho un cop</button>
2 <ul id="resultats"></ul>

1 const botoUnCop = document.getElementById("un-cop");
2 const resultats = document.getElementById("resultats");
3
4 // Definim la funció gestora en una variable/funció amb nom
5 function gestionarClicUnCop() {
6   const li = document.createElement("li");
7   li.textContent = "Acció executada!";
8   resultats.appendChild(li);
9
10  // Després del primer ús, eliminem el listener
11  botoUnCop.removeEventListener("click", gestionarClicUnCop);
12
13  // Opcional: canviem l'aspecte del botó
14  botoUnCop.textContent = "Ja està fet";
15  botoUnCop.disabled = true;
16 }
17
18 // Afegim el listener
19 botoUnCop.addEventListener("click", gestionarClicUnCop);
```

En aquest exemple:

1. El primer clic dispara `gestionarClicUnCop`, que afegeix un `<li>` i **es desregistra a si mateixa** amb `removeEventListener`.

Perquè `removeEventListener` funcioni, el navegador ha de poder **reconèixer la mateixa funció**. Per això sovint definim el gestor en una variable o funció amb nom, en comptes de posar una funció anònima directament dins d'`addEventListener`.

2. A partir del segon clic, el botó ja no té cap gestor per al `click`, així que **no passa res**.

Per què no funciona amb funcions anònimes?

Error típic:

```
1 const boto = document.getElementById("boto");
2
3 boto.addEventListener("click", function () {
4   console.log("Clic!");
5 });
6
7 // Més tard...
8 boto.removeEventListener("click", function () {
9   console.log("Clic!");
10 });
```

Això **no elimina** el *listener*. Tot i que el codi de la funció sembla igual, per al navegador són **dues funcions diferents** (dues funcions anònimes creades en moments diferents). Per això és important **reutilitzar la mateixa referència**:

```
1 function gestionarClic() {
2   console.log("Clic!");
3 }
4
5 boto.addEventListener("click", gestionarClic);
6
7 // Més tard...
8 boto.removeEventListener("click", gestionarClic); // Ara sí
```

2.3.3 Propagació d'esdeveniments i `stopPropagation`

Als apunts d'esdeveniments ja vam veure que, quan es produeix un esdeveniment en un element concret del DOM, **no es queda només en aquell node**: per defecte, l'esdeveniment es **propaga cap als elements "pare"**. És el que s'anomena **propagació** (o **bubbling**): primer es dispara al node on ha passat (el **target**), després al seu contenidor, en acabat al contenidor del contenidor, i així successivament fins arribar a `<body>` o `<html>`.

Ara que ja estem posant *listeners* sobre elements niats (divs, botons, targetes...), és important entendre aquesta propagació per poder decidir **quan volem que un clic "pugi" als pares i quan no**.

Per aturar aquesta cadena de propagació fem servir el mètode **`event.stopPropagation()`** dins del gestor de l'esdeveniment.

Exemple: un botó dins d'un `<div>`

Imaginem que tenim un `<div>` que ocupa una zona de la pàgina i, a dins, un botó. Volem que:

- si cliquem **directament el div**, es mostri un missatge ("Has clicat la caixa");

- si cliquem **el botó**, es mostri només el missatge del botó, **sense executar el clic del div**.

```
1 <div id="caixa">
2   <p>Soc una caixa. Clica'm o clica el botó de dins:</p>
3   <button id="boto">Clica el botó</button>
4 </div>
```

Primera versió (sense aturar la propagació):

```
1 const caixa = document.getElementById("caixa");
2 const boto = document.getElementById("boto");
3
4 caixa.addEventListener("click", function () {
5   console.log("Has clicat la CAIXA");
6 });
7
8 boto.addEventListener("click", function () {
9   console.log("Has clicat el BOTÓ");
10 });
```

Què passa aquí?

- Si cliquem el **div** (fora del botó):
 - es dispara el `click` sobre `caixa` → missatge “Has clicat la CAIXA”.
- Si cliquem el **botó**:
 - primer es dispara el `click` sobre el botó (`boto`) → “Has clicat el BOTÓ”;
 - després l'esdeveniment **es propaga al pare** (`caixa`) → també es mostra “Has clicat la CAIXA”.

És a dir, en clicar el botó veiem **dos missatges** perquè el `click` “puja” del botó cap al div.

Aturar la propagació amb `'event.stopPropagation()'`

Si volem que, en clicar el botó, **només** s'executi el seu propi gestor i **no** el del div, hem d'aturar la propagació dins del *listener* del botó:

```
1 const caixa = document.getElementById("caixa");
2 const boto = document.getElementById("boto");
3
4 caixa.addEventListener("click", function () {
5   console.log("Has clicat la CAIXA");
6 });
7
8 boto.addEventListener("click", function (event) {
9   // Aturem que el clic "puji" fins al div pare
10  event.stopPropagation();
11
12  console.log("Has clicat el BOTÓ (sense activar el div)");
13 });
```

Ara el comportament és:

- Clic sobre el **div** (fora del botó) →
 - → només es mostra “Has clicat la CAIXA”.
- Clic sobre el **botó** →

- → el navegador crida el *listener* del botó, que fa `event.stopPropagation()`.
- → **no** s'executa el listener del div.
- → només es mostra “Has clicat el BOTÓ (sense activar el div)”.